

Duale Hochschule Baden-Württemberg Mannheim

Studienarbeit

Übersetzung von Vorlesungsunterlagen in TypeScript – Theoretische Informatik III

Studiengang Informatik

Studienrichtung Angewandte Informatik

Verfasser:	Ertan Ertas und Christian Strerath
Matrikelnummer:	9390224, 4965013
Kurs:	TINF23AI2
Studiengangsleiter:	Prof. Dr. Holger Hofmann
Betreuer:	Prof. Dr. Karl Stroetmann
Bearbeitungszeitraum:	15.10.2025 - 14.04.2026

Erklärung

Wir versichern hiermit, dass wir unsere Studienarbeit mit dem Thema: „Übersetzung von Vorlesungsunterlagen in TypeScript – Theoretische Informatik III“ selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt haben.

Mannheim, 5. März 2026

Ertan Ertas

Kaiserslautern, 5. März 2026

Christian Strerath

Kurzfassung (Abstract)

Auf Deutsch

In English

Inhaltsverzeichnis

1	Einleitung	1
1.1	Problemstellung	1
1.2	Zielsetzung	3
1.3	Methodik und Abgrenzung	3
2	Theoretische Grundlagen	5
A	Anhang	6
A.1	Verzeichnis der portierten Lehrnotebooks	6

Abbildungsverzeichnis

Tabellenverzeichnis

1	Übersicht der übersetzten Notebooks im Repository	6
---	---	---

List of Listings

1	Mengen in Python: Das erwartete Verhalten	2
2	Mengen in TypeScript: Das fehlerhafte Verhalten bei Tupeln	2

Glossar

Repository Ein zentraler, oft versionierter Speicherort für Quellcode und digitale Dokumente, der es ermöglicht, Änderungen am Code historisch nachzuvollziehen und gemeinsam an Projekten zu arbeiten. 4

1 Einleitung

Die Theoretische Informatik bildet ein wichtiges Fundament im Informatikstudium. Themen wie Automaten oder formale Sprachen können anfangs jedoch sehr abstrakt wirken. Um diese theoretischen Konzepte greifbarer zu machen, kommen in der Vorlesung „Theoretische Informatik III“ interaktive Notebooks zum Einsatz. Dort wird erklärender Text direkt mit ausführbarem Programmcode gemischt. Bislang basieren diese Notebooks auf der Programmiersprache Python. Studierende können so theoretische Algorithmen direkt ausprobieren, verändern und live nachvollziehen.

Diese Arbeit befasst sich mit der Übertragung dieser Lehrinhalte in die Programmiersprache TypeScript. Der Fokus liegt dabei auf einer Portierung, welche nicht nur die Syntax übersetzt, sondern die Konzepte der formalen Sprachen durch verbindliche Datenstrukturen explizit modelliert.

1.1 Problemstellung

Ausgangspunkt dieser Arbeit sind interaktive Python-Notebooks, die Konzepte der theoretischen Informatik demonstrieren. Werden diese Lehrmaterialien in das TypeScript-Ökosystem übertragen, reicht es jedoch nicht aus, lediglich die Syntax der Sprache zu übersetzen. Es entstehen fundamentale technische Hürden, da beide Sprachen Daten auf unterschiedliche Weise verarbeiten. Folgende zwei Problemfelder stehen dabei im Fokus:

Zu späte Fehlererkennung bei verschachtelten Daten

In Python haben Variablen keine festen Typen. Das führt oft dazu, dass Programmierfehler erst auffallen, wenn der Code ausgeführt wird und plötzlich abstürzt. Zwar gibt es externe Werkzeuge, um Datentypen in Python nachträglich zu prüfen, diese sind jedoch nicht nativ in die Sprache integriert und stoßen gerade in der interaktiven Umgebung der Notebooks oft an ihre Grenzen.

In der Vorlesung arbeiten wir ständig mit tief verschachtelten Objekten, beispielsweise Datenstrukturen, bei denen eine Liste wieder viele weitere Objekte und Listen enthält. Fehlt einem dieser verschachtelten Elemente versehentlich eine wichtige Eigenschaft, merkt Python das erst, wenn der Code im Hintergrund genau dieses fehlerhafte Element aufruft. Wir benötigen stattdessen eine Sprache, die uns schon während des Tippens garantiert, dass unsere Datenstrukturen über alle Ebenen hinweg fehlerfrei und vollständig aufgebaut sind.

Das unerwartete Verhalten von Mengen (Sets)

Das gravierendste Hindernis bei der Übersetzung zeigt sich jedoch bei der Arbeit mit Mengen (Sets). In der theoretischen Informatik ist es unumgänglich, berechnete Zustände in Mengen zu speichern, um Duplikate zu vermeiden oder später zu prüfen, ob ein Zustand bereits erreicht wurde.

In Python funktioniert das exakt so, wie man es intuitiv erwartet. Fügen wir ein Tupel in eine Menge ein und fragen danach ab, ob ein Tupel mit denselben Zahlen enthalten ist, liefert Python das erwartete Ergebnis:

```
1  menge = set()
2  menge.add((1, 2))
3
4  # Die Menge enthält das Tupel sichtbar
5  print(menge)           # Ausgabe: {(1, 2)}
6
7  # Die Abfrage liefert das erwartete Ergebnis
8  print((1, 2) in menge) # Ausgabe: True
```

Listing 1: Mengen in Python: Das erwartete Verhalten

Ein naiver Übersetzungsversuch nach TypeScript führt hier jedoch zu einem fundamentalen Bruch in der Logik. Zwar erlaubt uns die Sprache, im Code ganz genau festzulegen, dass unsere Menge ausschließlich Tupel aus exakt zwei Zahlen aufnehmen darf. Führen wir diesen Code jedoch aus und fragen das zuvor hinzugefügte Element ab, erhalten wir plötzlich ein falsches Ergebnis:

```
1  // Wir definieren den Typ Tuple, der zwei Zahlen akzeptiert.
2  type Tuple = [number, number];
3
4  // Die Menge soll nur diese Tupel akzeptieren
5  const menge = new Set<Tuple>();
6  menge.add([1, 2]);
7
8  // Die Menge enthält das Element sichtbar
9  console.log(menge);           // Ausgabe: Set(1) { [ 1, 2 ] }
10
11 // Die Abfrage liefert ein ungewöhnliches Ergebnis
12 console.log(menge.has([1, 2])); // Ausgabe: false
```

Listing 2: Mengen in TypeScript: Das fehlerhafte Verhalten bei Tupeln

Obwohl die Konsolenausgabe beweist, dass das Element `[1, 2]` offensichtlich in die Menge eingefügt wurde, behauptet TypeScript's Standard-Set in der

anschließenden Abfrage, es sei nicht vorhanden. Dieser zunächst unerklärliche Widerspruch führt bei einer direkten Portierung dazu, dass Logik-Abfragen fehlschlagen, Endlosschleifen entstehen und Algorithmen falsche Ergebnisse liefern.

1.2 Zielsetzung

Das übergeordnete Ziel dieser Arbeit ist es, die Lehrinhalte der Vorlesung „Theoretische Informatik III“ vollständig nach TypeScript zu übersetzen. Die neue Codebasis soll die bisherigen Python-Notebooks ersetzen und dabei die strikten Typvorgaben der neuen Sprache nutzen, um Fehler schon während des Schreibens zu verhindern.

Daraus leiten sich folgende konkrete Teilziele ab:

1. **Strukturierte Neuimplementierung:** Die Algorithmen der Vorlesung sollen vollständig in TypeScript portiert werden. Der fehleranfällige Verzicht auf feste Typen in Python wird dabei durch verbindliche Datenstrukturen und strikte Typvorgaben ersetzt.
2. **Lösung des Mengen-Problems:** Um die Algorithmen fehlerfrei auszuführen, muss das fehlerhafte Verhalten von TypeScripts Standard-Sets umgangen werden. Das zentrale technische Ziel ist daher die Entwicklung einer eigenen Datenstruktur, die sich beim Vergleichen von Inhalten exakt wie ein Python-Set verhält und auch tief verschachtelte Objekte verlässlich erkennt.
3. **Einheitliche Ausführungsumgebung:** Da die Notebooks lokal auf den Rechnern der Studierenden ausgeführt werden, muss eine standardisierte Umgebung geschaffen werden, die unabhängig vom Betriebssystem identisch funktioniert. Ein wichtiges technisches Teilziel ist es dabei, die Hintergrundprozesse der Notebooks so anzupassen, dass TypeScripts strenge Prüfregeln auch bei der interaktiven, schrittweisen Ausführung zuverlässig greifen. Der Einrichtungsaufwand soll durch eine klare Anleitung minimal gehalten werden.

1.3 Methodik und Abgrenzung

Die methodische Durchführung dieser Arbeit folgt einem fokussierten Ansatz. Der Umfang der Portierung beschränkt sich auf die nummerierten Lehrnotebooks der Vorlesung sowie deren unmittelbare technische Abhängigkeiten. Übungsnotebooks, die primär dem unbetreuten Selbststudium dienen, sind explizit vom Umfang dieser Arbeit ausgeklammert. Die genaue Auswahl der zu übersetzenden

Artefakte erfolgte in Abstimmung mit dem Betreuer und ist im Anhang (siehe Abschnitt A.1) tabellarisch aufgeführt.

Ein zentraler methodischer Aspekt betrifft die didaktische Aufbereitung: Die begleitenden Erklärtexpte der Notebooks wurden nicht isoliert übersetzt, sondern inhaltlich an die neuen Strukturen von TypeScript angepasst. Dabei wurde bewusst darauf verzichtet, die neuen Konzepte kontinuierlich mit der alten Python-Implementierung zu vergleichen. Die Notebooks sollen als eigenständiges Lehrmaterial funktionieren, das die theoretischen Konzepte nativ und in sich geschlossen vermittelt.

Auf technischer Ebene war die Minimierung von Systemabhängigkeiten ein wesentliches Entwurfsziel. Insbesondere für die grafische Visualisierung von Automaten und Graphen wurde darauf geachtet, keine komplexen lokalen Installationen von externer Software vorauszusetzen. Stattdessen kommen integrierbare, plattformunabhängige Lösungen zum Einsatz, um die Ausführung auf verschiedenen Betriebssystemen zu garantieren und den Einstieg für die Studierenden so barrierefrei wie möglich zu gestalten.

Um die Ergebnisse dieser Arbeit nachvollziehbar und frei zugänglich zu machen, wird der gesamte entwickelte Quellcode in einem öffentlichen Repository bereitgestellt. Die Implementierung ist unter der URL <https://github.com/cstreich/fomal-languages-typescript> im Verzeichnis TypeScript abrufbar.

2 Theoretische Grundlagen

A Anhang

A.1 Verzeichnis der portierten Lehrnotebooks

Die folgende Tabelle listet alle Jupyter-Notebooks auf, die im Rahmen dieser Arbeit selektiert und von Python nach TypeScript portiert wurden. Die Gliederung folgt der Kapitelstruktur der Originalvorlesung „Theoretische Informatik III“.

Tabelle 1: Übersicht der übersetzten Notebooks im Repository

Original-Kapitel	Notebook-Dateiname
Chapter 03	01-Ply-Scanning-Example.ipynb 02-Regex-Tutorial.ipynb 03-Exam-Evaluation.ipynb 04-Html2Text.ipynb
Chapter 04 & 05	01-NFA-2-DFA.ipynb 02-Test-NFA-2-DFA.ipynb 03-Regex-2-NFA.ipynb 04-Test-Regex-2-NFA.ipynb 05-DFA-2-RegExp.ipynb 06-Test-DFA-2-RegExp.ipynb 07-Minimize.ipynb 08-Test-Minimize.ipynb 09-Equivalence.ipynb 10-Test-Equivalence.ipynb
Chapter 06	01-Top-Down-Parser.ipynb 02-EBNF-Parser.ipynb
Chapter 07	01-Calculator.ipynb 02-Differentiator.ipynb 03-Interpreter.ipynb
Chapter 10	01-Conflicts.ipynb 02-Conflicts-Resolved.ipynb

Quelle: Eigene Darstellung.